



CORPORACIÓN UNIVERSITARIA DEL HUILA – CORHUILA

Taller: Creación de APIs Simples para Usuarios

Integrantes: Barrera Giraldo Harold Camilo

Facultad De Ingeniería, Corporación Universitaria del Huila-CORHUILA

Programación Móvil

Docente: Jesus Ariel Gonzalez Bonilla

03 de septiembre del 2025

Informe Reflexivo: Implementación de la API REST para Gestión de Usuarios

Introducción

La implementación de una API REST para la gestión de usuarios, utilizando tecnologías como NestJS, JWT y la librería Swagger, representó un hito fundamental en el entendimiento del desarrollo *backend* moderno. Este proceso no solo se enfocó en la funcionalidad CRUD (Crear, Leer, Actualizar, Eliminar) de los usuarios, sino que también profundizó en la **capa crítica de seguridad y documentación**. Las lecciones y los desafíos superados en esta etapa son esenciales para el desarrollo de futuras aplicaciones, especialmente en entornos móviles donde la fiabilidad y la estructura del *backend* son determinantes.

Lecciones Aprendidas y Consolidación de Conocimientos

El desarrollo de esta API proporcionó una inmersión profunda en varios pilares de la arquitectura de servicios web:

1. El Sistema de Autenticación con JWT y Passport

La principal lección giró en torno al uso de **JSON Web Tokens (JWT)** junto con la librería **PassportJS** en NestJS. Aprendí la función crítica de las clases y decoradores involucrados:

- **PassportStrategy**: Comprendí que esta es la clase base que define la lógica para la validación del token. La **JwtStrategy** es la encargada de extraer el token del encabezado, verificar su firma y tiempo de expiración (usando la clave secreta) y, finalmente, devolver el objeto payload del usuario a la aplicación.
- **@UseGuards(AuthGuard('jwt'))**: Esta línea de código se convirtió en el "interruptor de seguridad". Entendí que un **Guard (vigilante)** es un *middleware* que se ejecuta **antes** del método del controlador. Si la **JwtStrategy** falla al validar el token (por estar ausente, inválido o expirado), el *Guard* lanza automáticamente la respuesta **401 Unauthorized**, impidiendo que el código del *endpoint* protegido se ejecute. Esto garantiza la integridad de los datos.

2. Documentación Automática con Swagger

Descubrí la importancia de la documentación de API no solo como un entregable, sino como una **herramienta viva de desarrollo y depuración**. La implementación de Swagger, configurando el **DocumentBuilder** y usando decoradores como **@ApiTags()**, **@ApiOperation()**, y el crítico **@ApiBearerAuth()**,

me enseñó que la documentación puede ser generada directamente desde el código. Esto facilita la colaboración y el testeo, ya que la documentación y la API real siempre están sincronizadas.

Dificultades Encontradas y Soluciones Implementadas

Los desafíos enfrentados estuvieron directamente relacionados con la configuración de seguridad y las operaciones de datos:

1. Validación de Token en la Interfaz de Swagger

Dificultad: Inicialmente, tuve problemas para que el *endpoint* de verificación de token (`/api/auth`) fuera funcionalmente seguro. Incluso con el `AuthGuard` implementado, la ruta parecía devolver 200 OK sin ingresar un token. Además, la interfaz de Swagger no mostraba el botón para ingresar la cadena del token.

Solución: Tras una revisión exhaustiva, se identificaron y resolvieron dos problemas:

1. **Swagger:** Se corrigió la configuración en `main.ts` añadiendo el esquema completo de seguridad con `.addBearerAuth()` y la referencia 'access-token', lo cual forzó la aparición del botón **"Authorize"** en la interfaz.
2. **Seguridad Real:** Se verificó la correcta inclusión del **PassportModule** en el `AuthModule` y la coherencia de la clave secreta (`secretOrKey`) entre el `AuthModule` y la `JwtStrategy`. Esto aseguró que el *Guard* interceptara la solicitud, devolviendo el esperado **401 Unauthorized** cuando el token estaba ausente.

2. Cifrado de Contraseñas al Actualizar (Update)

Dificultad: Al implementar la funcionalidad de actualización de usuario (PUT/PATCH), me di cuenta de que si un usuario enviaba una nueva contraseña, esta se guardaba en la base de datos en **texto plano**, comprometiendo la seguridad.

Solución: Se implementó una lógica de *hashing* dentro del servicio (o el *DTO* de actualización, según el enfoque) que **verifica si el campo de contraseña está presente en la solicitud**. Si está presente, se aplica la función de cifrado de **bcrypt** antes de enviar los datos a la base de datos. Si el campo de contraseña no se incluye en el *payload* de actualización, simplemente se ignoran los datos, preservando el *hash* existente.

Importancia de un Backend Estructurado para Aplicaciones Móviles

El proceso de desarrollo de esta API subraya la necesidad crítica de un *backend* bien estructurado cuando se trabaja con aplicaciones móviles.

1. Fiabilidad y Rendimiento: Una arquitectura modular (como la proporcionada por NestJS) asegura que la lógica de negocio, la seguridad y la capa de datos estén separadas. Esto facilita la escalabilidad y la optimización del rendimiento. Las aplicaciones móviles dependen de respuestas rápidas; un *backend* estructurado minimiza los cuellos de botella.

2. Seguridad Inquebrantable: En el ecosistema móvil, la exposición de datos es un riesgo constante. Implementar JWT y *Guards* garantiza que: * No se transmiten credenciales en cada solicitud (solo el token). * Solo se puede acceder a las funcionalidades protegidas si el usuario está autenticado.

3. Mantenibilidad y Evolución: La claridad proporcionada por la documentación de Swagger y la estructura de NestJS (Controladores, Servicios, Módulos) permite a los desarrolladores *frontend* (móviles) consumir la API con total confianza, sabiendo qué datos esperar (gracias a los DTOs) y cómo interactuar con cada *endpoint*. Esto es fundamental para la evolución ágil de la aplicación. En resumen, un *backend* robusto y seguro es la base para cualquier experiencia móvil exitosa.